

High-Level Design

for

Design and Development of Identity Security Framework for EdTech

Jesteena Mary Oommen

23BCCED016

BCA in Cyber Security, Ethical Hacking & Digital Forensics with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

11-04-2026

Under the guidance of

Mr. Vikyath

Lecturer, Computer Science Department

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to



YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATTA, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction
 - 1.1 Purpose of the Document
 - 1.2 Scope
 - 1.3 System Overview
2. System Architecture
 - 2.1 High-Level Component Diagram
 - 2.2 Core Components
 - 2.3 Authentication & Identity Restoration Flow
3. Security Defense Mechanics
 - 3.1 Multi-Layer Threat Prevention
 - 3.2 Node Quarantine & Lockout Policies
 - 3.3 Quantum Guard (Audit Log Integrity)
4. Database Schema Overview
 - 4.1 Entity-Relationship Diagram
 - 4.2 Primary Tables
 - 4.3 Auxiliary Protection Tables
5. Integration & Deployment
 - 5.1 External Integrations
 - 5.2 Deployment Architecture
6. Compliance & Privacy
 - 6.1 FERPA Alignment
 - 6.2 Student Data Protection
7. Conclusion

1. Introduction

1.1 Purpose of the Document

This High-Level Design (HLD) document provides an architectural blueprint for the Identity Security Framework for EdTech. It describes the major system components, their interactions, security mechanisms, and data structures at a level suitable for stakeholders, architects, and implementers.

1.2 Scope

The HLD covers:

- The modular Flask-based web application architecture.
- Authentication methods (password, OAuth, WebAuthn) and the multi-vector identity restoration protocol.
- Security defenses: node quarantine, brute-force lockout, IP blacklisting, and quantum-resistant audit logging.
- Database schema overview for user identities, recovery fingerprints, and auxiliary protection tables.
- External integrations (Google OAuth, FIDO2) and deployment considerations.

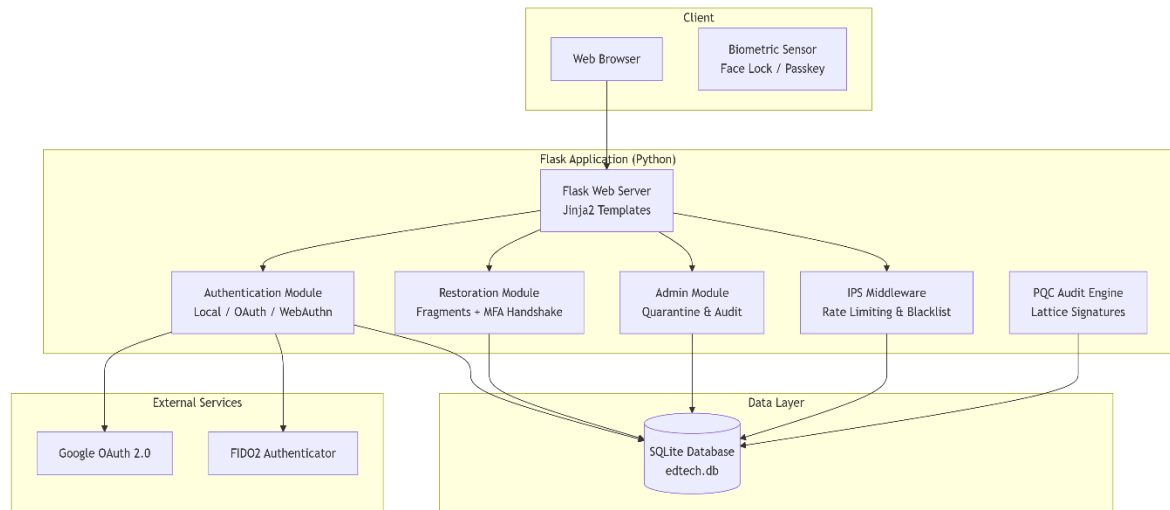
1.3 System Overview

The EdTech Identity Security Framework is a **high-fidelity web application** that replaces traditional password-only access with a zero-trust, passwordless-ready authentication system. It serves students, faculty, and administrators, ensuring that academic records are protected by:

- **Phishing-resistant WebAuthn (Passkeys / Face Lock)** as the primary modern authentication.
- **Fallback TOTP** for secondary MFA.
- **Combined Dual-Factor Handshake** for password restoration – requiring both password fragments and TOTP.
- **Post-quantum-simulated audit logs** for non-repudiation.

2. System Design

2.1 High-Level Component Diagram



2.2 Core Components

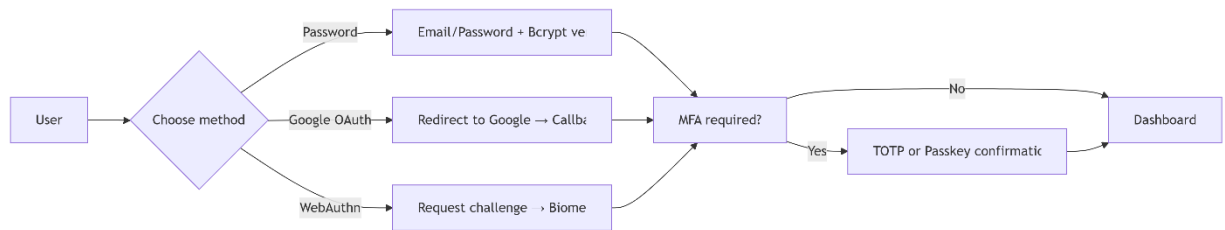
Component	Technology	Responsibility
Web Framework	Flask / Flask-Login	HTTP routing, session management, template rendering.
Database Interface	Flask-SQLAlchemy	ORM for <code>edtech.db</code> , schema migrations.
OAuth Engine	Authlib	Google OpenID Connect (student/teacher SSO).
TOTP Engine	PyOTP	6-digit time-based code validation for MFA.
WebAuthn Engine	FIDO2 / WebAuthn library	Passkey and biometric (Face Lock) registration & authentication.
Frontend	Jinja2 + Tailwind CSS	Cyber-Glass UI, responsive dashboards.
IPS Middleware	Custom <code>@app.before_request</code>	Rate limiting, IP blacklisting, brute-force detection.

PQC Audit	<code>lattice_crypto.py</code>	Simulated post-quantum signatures for audit log tamper detection.
------------------	--------------------------------	---

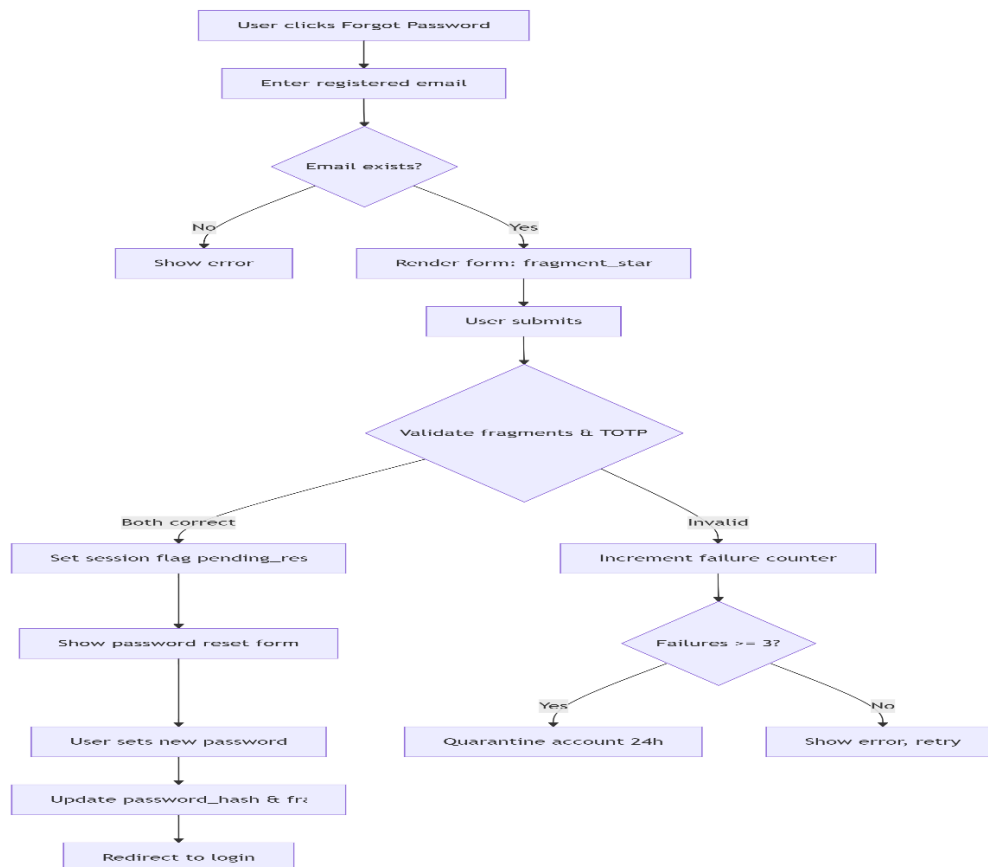
2.3 Authentication & Identity Restoration Flow

The system supports three authentication paths and one restoration path.

2.3.1 Primary Authentication Paths



2.3.2 Multi-Vector Identity Restoration (Forgot Password)



3. Security Defense Mechanics

3.1 Multi-Layer Threat Prevention

Threat	Mitigation	Implementation
Brute-force login	Account lockout	5 failures → locked 15 minutes
Restoration brute-force	Node quarantine	3 failures → locked 24 hours
SQL injection	Parameterized queries	SQLAlchemy ORM
XSS	Output escaping	Jinja2 autoescape + CSP
Credential stuffing	WebAuthn (passwordless)	FIDO2 prevents replay
Session hijacking	Secure cookies	Secure, HttpOnly, SameSite=Lax

3.2 Node Quarantine & Lockout Policies

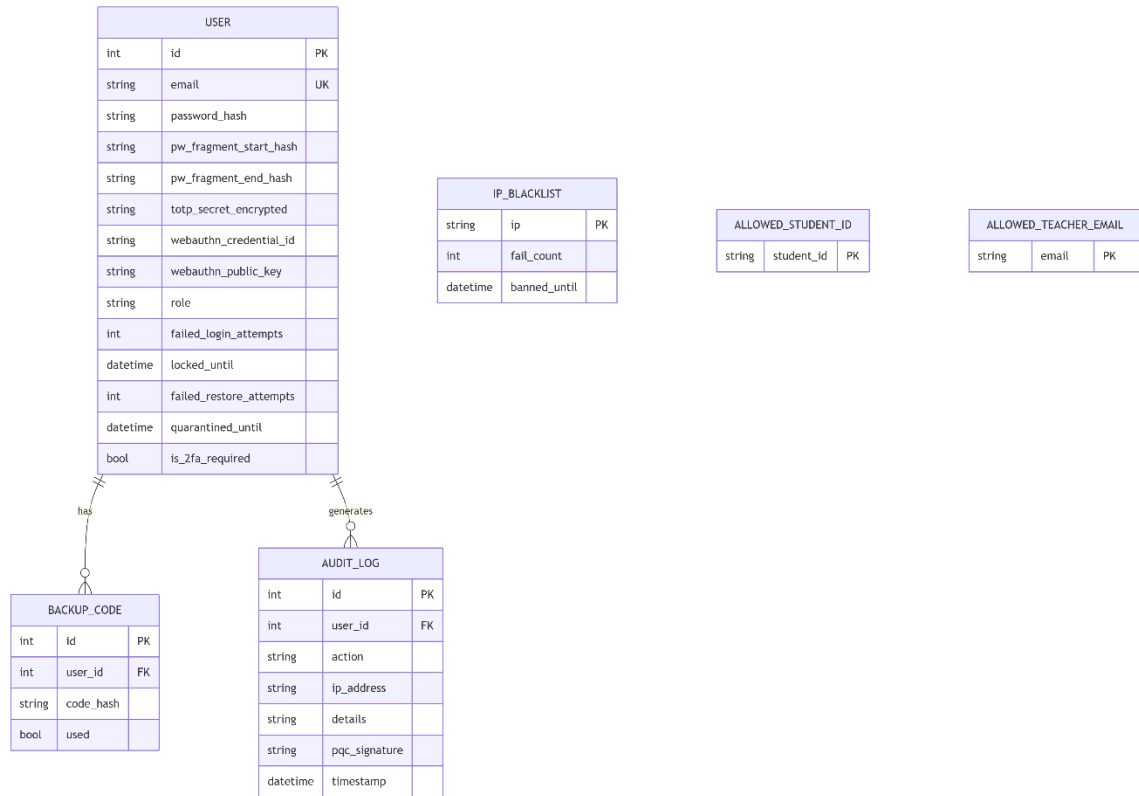
- **Login failures:** Counter per user. After 5 failed attempts within 15 minutes, account locked for 15 minutes.
- **Restoration failures (fragments + TOTP):** Separate counter. After 3 failures, account quarantined for 24 hours.
- **IP blacklisting:** After 10 failed requests from same IP across any account, IP banned for 1 hour (configurable).
- **Admin override:** Administrator can clear quarantine or blacklist via admin panel.

3.3 Quantum Guard (Audit Log Integrity)

- Every security event (login, logout, restoration attempt, role change, quarantine) is written to audit_logs table.
- Each log entry includes a lattice-based signature (simulated post-quantum) computed over the log data + timestamp.
- The signature is stored in a separate column. Any tampering with the log entry (even one bit) invalidates the signature, detectable by the admin audit viewer.
- Keys for signing are rotated daily; old signatures remain verifiable using historical keys stored offline.

4. Database Schema Overview

4.1 Entity-Relationship Diagram



4.2 Primary Users Table

Field	Type	Description
id	INTEGER PK	Auto-increment
email	TEXT UNIQUE	Login identifier
password_hash	TEXT	Bcrypt (rounds=12)
pw_fragment_start_hash	TEXT	Bcrypt of first 4 chars (rounds=10)
pw_fragment_end_hash	TEXT	Bcrypt of last 4 chars (rounds=10)
totp_secret_encrypted	TEXT	AES-128 Fernet

		encrypted
webauthn_credential_id	TEXT	Unique WebAuthn credential ID
webauthn_public_key	TEXT	Public key (base64)
role	TEXT	'Student', 'Faculty', 'Admin'
failed_login_attempts	INTEGER	Counter for login lockout
locked_until	DATETIME	Login lock expiry
failed_restore_attempts	INTEGER	Counter for restoration quarantine
quarantined_until	DATETIME	Restoration lock expiry

4.3 Auxiliary Protection Tables

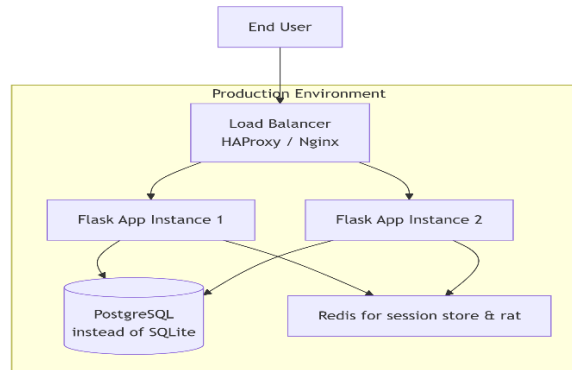
Table	Purpose	Key Fields
ip_blacklist	Block malicious IPs	ip, banned_until, fail_count
backup_codes	Emergency MFA fallback	user_id, code_hash (Bcrypt), used
audit_log	Immutable event log	action, user_id, pqc_signature, timestamp
allowed_student_ids	Whitelist for student registration	student_id (e.g., campus ID)
allowed_teacher_emails	Whitelist for faculty registration	email

5. Integration & Deployment

5.1 External Integrations

- **Google OAuth 2.0:** Provides institutional SSO for students/teachers using school-managed Google Workspace accounts. Callback endpoint: /auth/google/callback.
- **FIDO2 WebAuthn:** Works with platform authenticators (Windows Hello, Touch ID, Android biometrics). No external server required – browser handles cryptography.
- **Email service (optional):** For sending backup codes and quarantine notifications (SMTP or SendGrid).

5.2 Deployment Architecture (Production Recommendation)



Development (prototype) uses:

- Flask built-in server
- SQLite (file-based)
- No load balancing

6. Compliance & Privacy

6.1 FERPA Alignment

FERPA Requirement	How Framework Addresses
Access control to educational records	Role-based dashboards (Student, Faculty, Admin)
Audit of data access	audit_log table with PQC signatures
Secure authentication	WebAuthn + MFA + restoration handshake
Data minimization	Only necessary fields stored; fragments are hashed

6.2 Student Data Protection

- No plaintext passwords or fragments stored – only Bcrypt hashes.
- TOTP secrets encrypted at rest, Backup codes hashed with Bcrypt.
- IP blacklisting prevents automated attacks on student accounts.
- Quarantine protects against social engineering resets.

7. Conclusion

The High-Level Design (HLD) for the Identity Security Framework for EdTech defines a modern, zero-trust architecture that eliminates traditional password vulnerabilities. By integrating WebAuthn biometrics, a multi-vector restoration handshake (fragments + TOTP), and quantum-resistant audit logging, the framework provides a robust foundation for protecting student and faculty identities in learning management systems. The modular Flask-based design allows easy extension to other authentication methods (e.g., LDAP, SAML) and scalability to production deployments